

Declaración de variables

a	

b	

c	

arr	5000

y	1004
20	

x	1000
7	

a	

b	

c	

arr	5000

y	1004
20	

x	1000
7	



```
arr = (int *) malloc(MAX * sizeof(int));
```

Se reserva 3 espacios de tipo int en la memoria (12 bytes, 4 bytes cada uno), y arr guardara la dirección del primero de ellos.

a	1000

b	

c	

arr	5000

y	1004
20	

x	1000
7	

11 | a = &x;

a guarda la dirección de x

a	1000

b	1000

c	

arr	5000

y	1004
20	

x	1000
7	

12 | b = a;

b toma el mismo valor de a,
por lo que ahora b también
apunta a x

a	1000

b	1000

c	1004

arr	5000

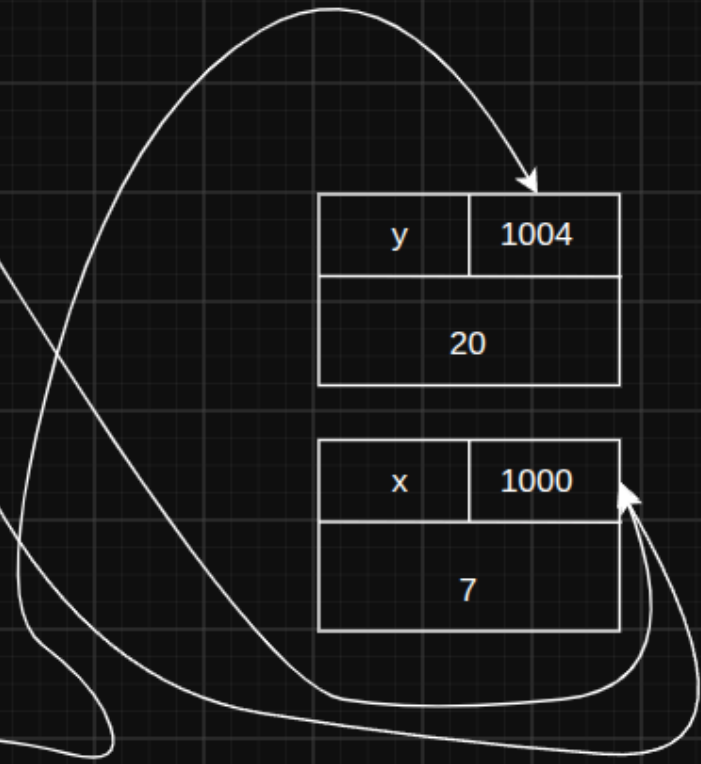
y	1004
20	

x	1000
7	

13

c = &y;

c toma la dirección de y



a	1000

b	1000

c	1004

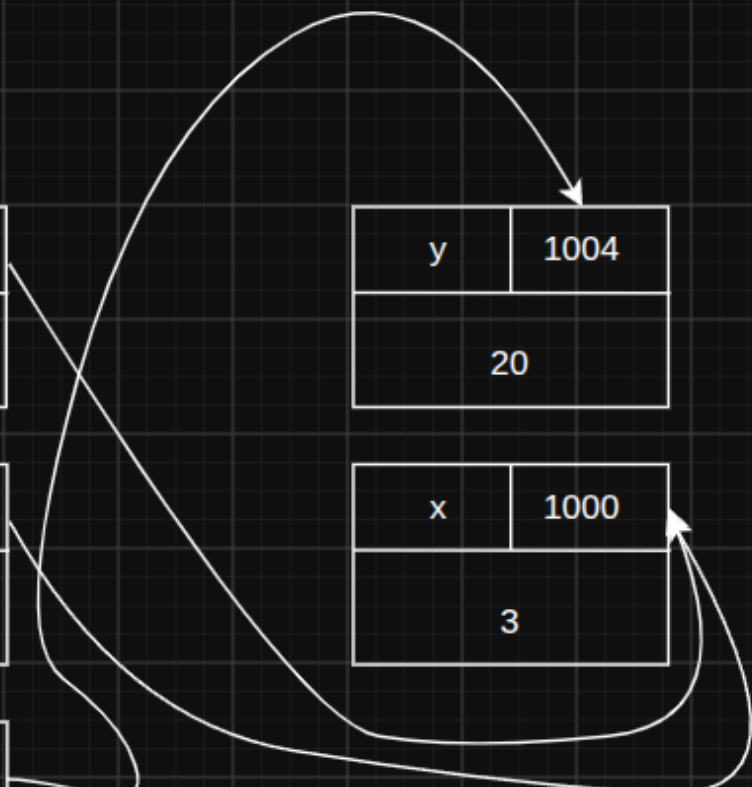
arr	5000

y	1004
20	

x	1000
3	

15 | *a = 3;

a sobrescribe a la casilla que apunta
(es decir, en x), 7 se borra y queda 3



a	1000

b	1000

c	1004

arr	5000

y	1004
	20

x	1000
	20

16 | *b = *c;

b copia el valor de "y" hacia "x" es decir ahora 3 pasa a ser 20

arr	5000

arr[0]	5000
	20

arr[1]	5004

arr[2]	5008

y	1004
	20

x	1000
	20

18 | `*(arr+1) = *c;`

`*(arr+1) = *c;`
recibe el valor de y (a través de c) es decir 20

arr	5000

y	1004
20	

arr[0]	5000
20	

x	1000
20	

arr[1]	5004

19 | `*(arr+2) = x + y;`

arr[2]	5008
40	

recibe la suma de $x + y$ es decir
 $(20 + 20) = 40$

a	1000

b	1000

c	1004

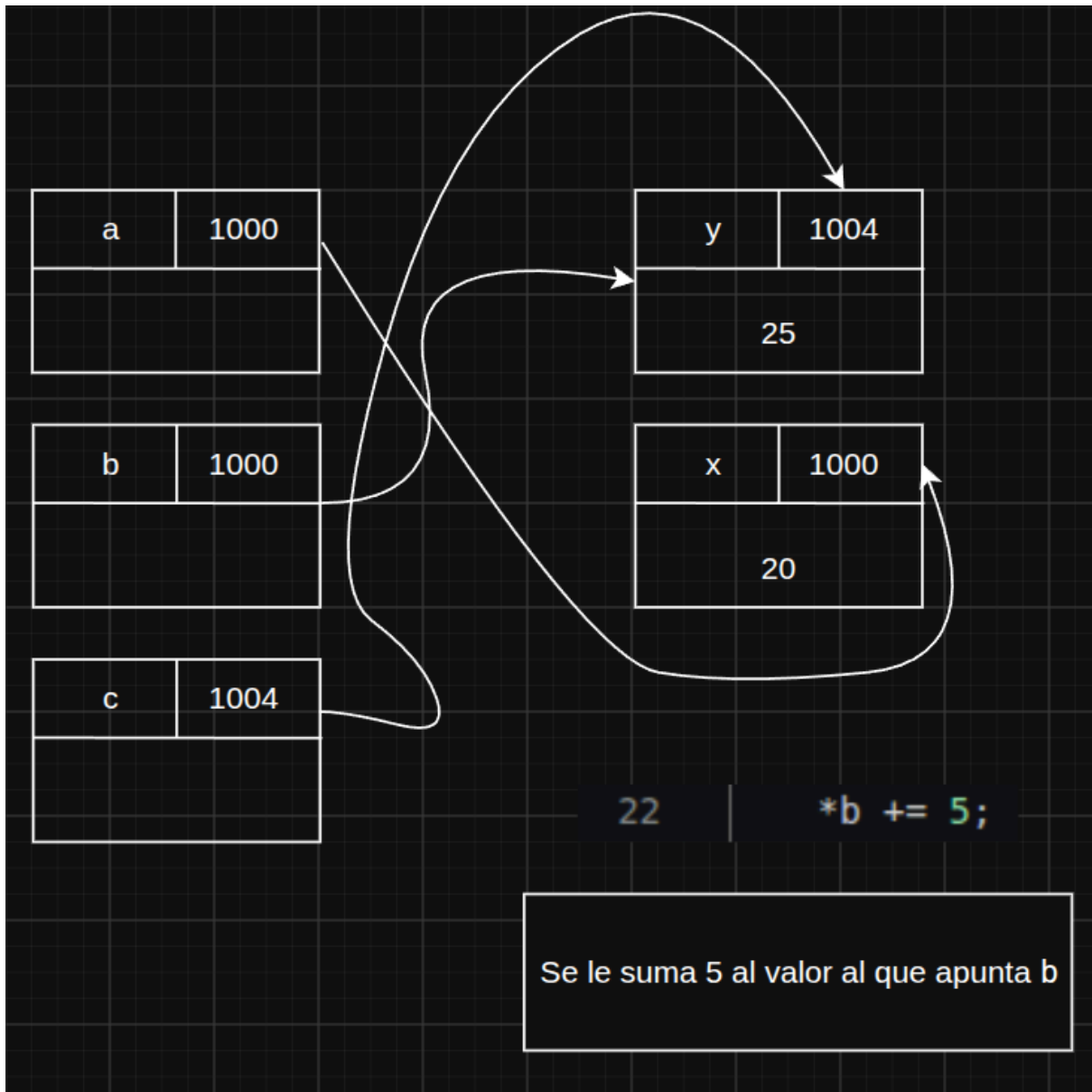
y	1004
20	

x	1000
20	

21

b = &y;

b deja de apuntar a x y ahora apunta a y



```
24 | printf("%d,%d,%d,%d,%d,%d", x, y, *arr, *(arr+1), *(arr+2), *a);
```

Finalmente se imprimen los valores por pantalla entregando los siguientes valores

```
20,25,20,20,40,20
```